# EGRefine: An Execution-Grounded Optimization Framework for Text-to-SQL Schema Refinement

Jiaqian Wang
Xidian University
Xi'an, Shaanxi, China
24151111272@stu.xidian.edu.cn

Yutao Qi*
Xidian University
Xi'an, Shaanxi, China
ytqi@xidian.edu.cn

Wenjin Hou
Xidian University
Xi'an, Shaanxi, China
houwenjinmail@gmail.com

Yu Pang
Xidian University
Xi'an, Shaanxi, China
24151213784@stu.xidian.edu.cn

Rui Yang
Xidian University
Xi'an, Shaanxi, China
ryang_2@stu.xidian.edu.cn

## ABSTRACT

Text-to-SQL enables non-expert users to query databases in natural language, yet real-world schemas often suffer from ambiguous, abbreviated, or inconsistent naming conventions that degrade model accuracy. Existing approaches treat schemas as fixed and address errors downstream. In this paper, we frame schema refinement as a constrained optimization problem: find a renaming function that maximizes downstream Text-to-SQL execution accuracy while preserving query equivalence through database views. We analyze the computational hardness of this problem, which motivates a column-wise greedy decomposition, and instantiate it as EGRefine: a four-phase pipeline that screens ambiguous columns, generates context-aware candidate names, verifies them through execution-grounded feedback, and materializes the result as non-destructive SQL views. The pipeline carries two structural properties: column-local non-degradation, ensured by the conservative selection rule in the verification phase, and database-level query equivalence, ensured by the view-based materialization phase. Together they make the resulting refinement safe by construction *at the column level*, with cross-column and prompt-level interactions handled empirically rather than analytically. Across controlled schema-degradation, real-world, and enterprise benchmarks, EGRefine recovers accuracy lost to schema naming noise where applicable and degrades gracefully when the underlying task exceeds current Text-to-SQL capabilities, with refined schemas transferring across model families to enable refine-once, serve-many-models deployment. Code and data are publicly available at https://github.com/ai-jiaqian/EGRefine.

## KEYWORDS

Text-to-SQL, Schema Refinement, Execution Feedback, Database Views, Optimization

**PVLDB Reference Format:**
Jiaqian Wang, Yutao Qi, Wenjin Hou, Yu Pang, and Rui Yang. EGRefine: An Execution-Grounded Optimization Framework for Text-to-SQL Schema Refinement. PVLDB, XX(X): XXX-XXX, 20XX.
doi:XX.XX/XXX.XX

*Corresponding author.

Proceedings of the VLDB Endowment, Vol. XX, No. X ISSN 2150-8097.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/ai-jiaqian/EGRefine.

## 1 INTRODUCTION

Text-to-SQL—the task of translating natural-language questions into executable SQL queries—has emerged as a pivotal technology for democratizing data access [22, 28]. Driven by rapid advances in large language models (LLMs), recent systems have achieved impressive performance on standard benchmarks [18, 33, 41, 47], with execution accuracies exceeding 85% on the widely used Spider dataset [54].

However, a persistent gap remains between benchmark performance and production reliability. Real-world databases are frequently characterized by inconsistent naming standards, ad-hoc abbreviations, and sparse documentation [15, 37]. Columns named `A2`, `nm`, `sal`, or `dt` carry little semantic information, forcing Text-to-SQL models to rely on fragile heuristic cues during schema linking [48]. Diagnostic evaluations confirm that even state-of-the-art systems suffer substantial accuracy degradation when schema names are perturbed or obfuscated [4, 26], establishing schema-level naming quality as a critical yet underexplored bottleneck for robust Text-to-SQL.

The prevailing response has been to treat schema quality as a fixed background condition and address its consequences *downstream*: constrained decoding improves syntactic validity but not naming ambiguity [39]; error-correction and self-refinement repair generated SQL after the fact [12, 31, 36], yet schema-linking errors rooted in opaque names persist; interactive clarification shifts the burden to users [14, 42, 53], impractical at scale; and recent ambiguity benchmarks probe query-level perturbations rather than schema-internal defects [3, 35, 38]. The schema itself—the root cause—remains untreated.

A small but growing line of work has begun to consider *upstream* schema-level interventions. Odin [43] recommends multiple SQL interpretations for ambiguous schemas at query time. CLEAR [55] provides a parser-independent disambiguation framework. While these efforts represent important steps, they share two fundamental

limitations. First, they operate at *query time*, incurring per-query overhead without producing durable schema-level improvements. Second, they lack a *grounded optimization objective*: the quality of a schema modification is assessed by linguistic plausibility rather than by its measurable effect on downstream task performance.

In this paper, we propose a fundamentally different approach. We observe that schema quality, viewed through the lens of Text-to-SQL, can be defined objectively as the downstream execution accuracy it induces. This allows us to formalize schema refinement as a constrained optimization problem: find a renaming of schema elements that maximizes execution accuracy while preserving query equivalence via SQL views. We analyze the computational hardness—the search space is exponential (Observation 1) and the constrained problem is at least NP-hard (Theorem 1)—and propose EGRefine (Execution-Grounded Refinement), a four-phase pipeline whose key innovation is *execution-grounded verification*: rather than relying on an LLM's semantic judgment to select column names, we use downstream SQL execution results as the selection signal, treating execution accuracy as a reward function for schema refinement. This applies execution feedback at a novel granularity—not to the generated query as in prior work [8, 11, 40], but to the schema itself.

Our contributions are as follows:

(1) **Formal Problem Framing.** We cast Text-to-SQL schema refinement as a constrained optimization problem with a task-grounded objective (Definition 2), analyze its hardness (§3.3), and establish two structural properties: *column-local non-degradation* (Proposition 1) and *database-level query equivalence* (Proposition 2).
(2) **The EGRefine Framework.** We operationalize this as a four-phase pipeline—heuristic screening (§4.1), candidate generation (§4.2), execution-grounded verification (§4.3), and non-destructive VIEW synthesis (§4.4)—that is model-agnostic: any downstream Text-to-SQL system benefits from the refined schema unmodified.
(3) **Comprehensive Empirical Evaluation.** We evaluate on three benchmarks: Dr.Spider [4] (controlled degradation), BIRD [26] (natural schemas), and BEAVER [10] (applicability boundary). Ablations show execution-grounded verification significantly outperforms pure LLM selection, regressions are rare, benefits amplify for weaker models, and a coverage–improvement linearity (§6) makes impact predictable at deployment.

## 2 RELATED WORK

We review three lines of research relevant to our work: Text-to-SQL systems, schema ambiguity and robustness, and execution-guided methods.

### 2.1 Text-to-SQL Systems

Text-to-SQL has progressed through several paradigm shifts: early sequence-to-sequence generation [56] and schema-aware encoders [27, 48]; constrained decoding for syntactic validity [39]; and decoupled schema-linking-then-parsing architectures [24]. The advent of LLMs shifted the dominant paradigm toward prompt-based methods such as DIN-SQL [33] (decomposed prompting), DAIL-SQL [18] (token-efficient skeleton-based prompts), and multi-agent systems including MAC-SQL [47] and CHESS [41]; fine-tuned variants such as CodeS [25] also appear in this landscape. Recent work [30] questions whether explicit schema linking is needed at all when LLMs have sufficient context; in either regime, schema-side quality remains a precondition for accurate generation. Comprehensive surveys include [20, 22, 28, 52].

All of the above treat the database schema as a fixed input. EGRefine is orthogonal: it operates as an upstream preprocessing layer that refines the schema before any Text-to-SQL system is invoked, composable with any of the above without modification.

### 2.2 Schema Ambiguity and Robustness

*2.2.1 Robustness Evaluation.* Earlier benchmarks established schema-side brittleness: Spider-Syn [16] showed accuracy collapses under synonym substitution that breaks lexical matching between question tokens and schema names, and Spider-DK [17] extended this to domain-knowledge generalization. Dr.Spider [4] subsequently introduced 17 perturbation types—including schema-level modifications—and showed substantial accuracy drops even for the strongest models. Fürst et al. [15] evaluated robustness to data model variations using real user queries, while Renggli et al. [37] identified fundamental challenges in evaluating Text-to-SQL under realistic conditions. These studies diagnose rather than resolve schema-side brittleness.

*2.2.2 Query-Level Ambiguity.* A separate line of work addresses ambiguity in the *question* rather than the *schema* [3, 35, 38], including interactive disambiguation that solicits user clarification [14, 34, 42] and detection of unanswerable questions [46]. These are complementary: we address ambiguity in the schema, not the question.

*2.2.3 Schema-Level Interventions.* Several benchmarks acknowledge schema quality as a real-world challenge: BIRD [26] provides supplementary knowledge descriptions alongside schemas; Spider 2.0 [23] features enterprise-scale schemas with over 1,000 columns and domain-specific abbreviations; BEAVER [10] provides enterprise data warehouse schemas with extremely low Text-to-SQL baselines; and BenchPress [51] addresses enterprise schema ambiguity through human-in-the-loop annotation.

Two recent systems directly tackle schema-level disambiguation. Odin [43] recommends multiple SQL candidates based on alternative schema interpretations and learns from user feedback. CLEAR [55] provides a parser-independent framework that generates and selects among candidate SQL interpretations. Chen et al. [9] address schema linking uncertainty through adaptive abstention. Closer to our setting, SNAILS [29] targets offline schema-level intervention: a classifier finetuned on ∼17K human labels flags low-naturalness identifiers, which a single-pass LLM call expands and exposes as views. Conceptually, SNAILS implements a subset of EGRefine's pipeline—a naturalness classifier as a Phase 1 analog and a single-pass expander as a Phase 2 analog—without verification or non-degradation guarantees. EGRefine differs on three axes: *target*—it optimizes ExAcc directly, not naturalness (a weakly-correlated proxy, as SNAILS reports); *verification*—Phase 3 selects among $k$ candidates by execution rather than committing

one unchecked expansion; and *non-degradation*—the conservative rule ($\tau_{\min}$) keeps original names when no gain is verified, which SNAILS lacks. We compare against SNAILS directly (§5.2, Table 1): its released naturalness classifier screens columns and Qwen3.5-27B generates replacement names, holding Phases 1–2 fixed so the comparison isolates execution-grounded selection. Our **LLM-Direct** baseline (§5.1) additionally probes the verification-free paradigm under LLM-only screening. The remaining methods (Odin, CLEAR, Chen et al.) operate at *query time* and judge schema modifications by linguistic plausibility or user feedback, whereas EGRefine refines *once* offline against a grounded, execution-tied objective.

## 2.3 Execution-Guided Methods

In Text-to-SQL, execution feedback has been used to constrain or re-rank candidate queries [39], iteratively repair SQL [12, 31], and drive self-correction loops in multi-agent frameworks [5, 47]. Beyond SQL, execution-based verification is effective in general code generation, including dual-execution agreement [8], self-debugging from execution traces [11], and verbal reinforcement learning over execution feedback [40].

Our work applies execution feedback at a different granularity: prior methods improve the *generated output* while holding the schema fixed, whereas EGRefine improves the *input representation* itself, yielding a durable refined schema that benefits subsequent queries drawn from the same workload. Among closely related approaches, EGRefine is the only one that is simultaneously *upstream*, *offline*, *execution-grounded*, *model-agnostic*, and *non-destructive*: the downstream methods PICARD [39] and DART-SQL [31] satisfy none of the first four, while the upstream query-time systems Odin [43] and CLEAR [55] are model-agnostic and non-destructive but neither offline nor execution-grounded.

**Workload-driven physical design.** Self-tuning databases tune physical structures such as indexes [6, 44], materialized views [2], and configuration knobs [45] against a representative workload, re-tuning on drift [7, 32]. EGRefine applies this to the *logical* (naming and view) layer rather than physical access paths, with Phase 3 verification (§4.3) as a what-if interface [44] grounded in real execution.

# 3 PROBLEM FORMULATION

In this section, we formalize schema refinement as a constrained optimization problem, analyze its computational hardness, and present our decomposition strategy with associated structural properties.

## 3.1 Preliminaries and Notation

**Database Schema.** A relational database is described by a schema $\mathcal{S} = (\mathcal{T}, \mathcal{C}, \mathcal{F})$, where $\mathcal{T} = \{t_1, \ldots, t_n\}$ is the set of tables, $\mathcal{C} = \{c_1, \ldots, c_m\}$ is the set of columns, and $\mathcal{F} \subseteq \mathcal{C} \times \mathcal{C}$ encodes foreign-key relationships. Each column $c \in \mathcal{C}$ belongs to a unique table $\text{tab}(c) \in \mathcal{T}$ and carries a *surface name* $\text{name}(c)$—the identifier visible to downstream systems. We define the *scope* of a column $c$ as the set of columns that share its naming namespace:

$$\text{scope}(c) = \{c' \in \mathcal{C} \mid \text{tab}(c') = \text{tab}(c)\} \cup \{c' \mid (c, c') \in \mathcal{F} \text{ or } (c', c) \in \mathcal{F}\}. \quad (1)$$

Scope includes same-table columns (where SQL forbids duplicate column names) and FK-related columns across tables (where duplicates create joining ambiguity). Cross-table homonyms without FK relationships are permitted by SQL and left untouched (§4.1).

**Text-to-SQL Task.** A Text-to-SQL model $M$ takes a natural-language question $nl$ and a schema $\mathcal{S}$ as input, and produces a predicted SQL query $M(nl, \mathcal{S})$. Performance is measured by *execution accuracy* (ExAcc): given a benchmark $Q = \{(nl_j, sql_j^*)\}_{j=1}^{|Q|}$ of question–gold-SQL pairs, we define

$$\text{ExAcc}(M, \mathcal{S}, Q) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \mathbb{1}\big[\text{exec}(M(nl_j, \mathcal{S})) = \text{exec}(sql_j^*)\big], \quad (2)$$

where $\text{exec}(\cdot)$ denotes the result set obtained by executing a query on the underlying database.

## 3.2 Schema Refinement as Optimization

We adopt a *task-grounded* perspective: schema quality is determined solely by its effect on downstream Text-to-SQL performance, avoiding subjective ambiguity taxonomies.

*Definition 1 (Refinement Mapping).* A *refinement mapping* $r : \mathcal{C} \to \Sigma^*$ assigns a (possibly new) surface name $r(c)$ to each column $c \in \mathcal{C}$, where $\Sigma^*$ is the set of valid SQL identifiers. The identity mapping $r_{\text{id}}$ corresponds to the unchanged schema; we write $r(\mathcal{S})$ for the schema obtained by applying $r$. A refinement is *admissible* if (i) no two columns in the same scope receive identical names, and (ii) only surface names are modified (data types, constraints, foreign keys, and table structure remain unchanged). We denote the set of admissible refinements by $\mathcal{R}(\mathcal{S})$.

*Definition 2 (Schema Quality).* Given $\mathcal{S}$, a set of Text-to-SQL models $\mathcal{M} = \{M_1, \ldots, M_l\}$, and a query set $Q$:

$$\text{Quality}(\mathcal{S}, \mathcal{M}, Q) = \frac{1}{|\mathcal{M}|} \sum_{M_j \in \mathcal{M}} \text{ExAcc}(M_j, \mathcal{S}, Q). \quad (3)$$

Averaging over multiple models ensures the metric reflects general schema clarity rather than single-system idiosyncrasies.

*Problem 1 (Optimal Schema Refinement).* Given $\mathcal{S}$, $\mathcal{M}$, and $Q$, find:

$$r^* = \arg \max_{r \in \mathcal{R}(\mathcal{S})} \text{Quality}\big(r(\mathcal{S}), \mathcal{M}, Q\big), \quad (4)$$

subject to: for every SQL query $q'$ over $r(\mathcal{S})$, $\text{exec}(q', r(\mathcal{S})) = \text{exec}(q, \mathcal{S})$ for the semantically equivalent query $q$ over $\mathcal{S}$.

The equivalence constraint ensures refinement is non-destructive: the original database remains untouched. Proposition 2 shows this is satisfied by construction when the refined schema is materialized as SQL views over the original tables.

*Remark 1.* A column $c$ is *refinement-improvable* iff some alternative name $c'$ strictly increases Quality when it replaces $\text{name}(c)$.

## 3.3 Hardness Analysis

Observation 1 (Exponential Search Space). *Let $|\mathcal{C}| = m$ and suppose each column has $k$ candidates (including the original). Then $|\mathcal{R}(\mathcal{S})| = O(k^m)$: for $m$=100 and $k$=3, $|\mathcal{R}(\mathcal{S})| > 5 \times 10^{47}$, rendering exhaustive search infeasible.*

To establish hardness rigorously, we introduce a constrained decision variant isolating the combinatorial core.

*Definition 3 (Constrained-Refinement-Decision (CRD)).* Given a schema $\mathcal{S}$ with column set $C = \{c_1, \ldots, c_n\}$, within-scope conflict relation $E \subseteq C \times C$, candidate lists $\mathcal{L}(c_i) \subseteq \Sigma^+$ ($|\mathcal{L}(c_i)| \geq 1$), and a forced-rename subset $C^\dagger \subseteq C$, does there exist an assignment $r : C \to \Sigma^+$ satisfying (i) $r(c_i) \in \mathcal{L}(c_i)$ for all $i$; (ii) $r(c_i) \neq r(c_j)$ for all $(c_i, c_j) \in E$; and (iii) $r(c_i) \neq c_i^0$ for all $c_i \in C^\dagger$?

Theorem 1 (NP-hardness). *The CRD problem is NP-hard. The optimization version of Problem 1—maximizing* Quality *over assignments satisfying the same constraints—is therefore also NP-hard.*

Proof. We reduce from List-Coloring, NP-complete for general graphs [21]: given $G = (V, E_G)$ with color lists $L(v) \subseteq \mathbb{N}$, decide whether a proper coloring $\phi$ with $\phi(v) \in L(v)$ and $\phi(u) \neq \phi(v)$ for all $(u, v) \in E_G$ exists. Map each $v_i$ to a column $c_i$ in one shared scope, set $E = E_G$, $\mathcal{L}(c_i) = \{\sigma(\ell) : \ell \in L(v_i)\}$ for an injective $\sigma : \mathbb{N} \to \Sigma^+$, and $C^\dagger = \emptyset$; the construction is polynomial. Then $r(c_i) = \sigma(\phi(v_i))$ is a valid CRD assignment iff $\phi$ is a proper coloring (by injectivity of $\sigma$; condition (iii) is vacuous). The optimization version of Problem 1 contains CRD as a feasibility sub-problem, hence is at least as hard. □

*Remark 2 (Scope of the Hardness Result).* Theorem 1 captures hardness from the combinatorial structure of constrained candidate assignment, not from properties of Quality. Inapproximability results exploiting structure of Quality itself remain open.

## 3.4 Decomposition and Structural Properties

Since exact optimization is intractable (Observation 1, Theorem 1), we decompose the joint problem into independent per-element subproblems, analogous to coordinate descent.

*3.4.1 Element-Wise Refinement.* For each column $c_i \in C$, let $Q(c_i) = \{(nl, sql^*) \in Q \mid c_i \text{ appears in } sql^*\}$ denote the queries referencing $c_i$. We solve the following subproblem independently for each $c_i$ identified as a refinement candidate (§4):

$$c_i^* = \arg \max_{c' \in \text{Cand}(c_i) \cup \{c_i^0\}} \text{Quality}\big(\mathcal{S}[c_i \to c'],\ \mathcal{M},\ Q(c_i)\big), \quad (5)$$

where $c_i^0 = \text{name}(c_i)$ is the original name, $\text{Cand}(c_i)$ is the set of $k$ candidates from the proposal module, and $\mathcal{S}[c_i \to c']$ denotes $\mathcal{S}$ with the substitution $\text{name}(c_i) \leftarrow c'$.

*3.4.2 Conservative Selection Rule.* To prevent regressions, we adopt a conservative policy: a column is renamed only if the best candidate strictly improves on the original. Let

$$\Delta_i = \max_{c' \in \text{Cand}(c_i)} \text{Quality}(\mathcal{S}[c_i \to c'], \mathcal{M}, Q(c_i)) - \text{Quality}(\mathcal{S}, \mathcal{M}, Q(c_i)) \quad (6)$$

denote the quality gain of the best candidate over the original. The conservative selection is:

$$c_i^* = \begin{cases} \arg\max_{c'} \text{Quality}(\cdot), & \text{if } \Delta_i > 0; \\ c_i^0, & \text{otherwise.} \end{cases} \quad (7)$$

This rule yields a local guarantee at the per-column level.

Proposition 1 (Per-Column Local Non-Degradation). *Consider the conservative selection rule* (7) *applied to a column $c_i$, holding all other columns fixed. Let $\mathcal{S}_{c_i \to c_i^0}$ denote the schema with $c_i$ keeping its original name, and $\mathcal{S}_{c_i \to c_i^*}$ the schema after applying the rule's selected name. Then on the column-local query subset $Q(c_i)$:*

$$\text{Quality}(\mathcal{S}_{c_i \to c_i^*}, \mathcal{M}, Q(c_i)) \ \geq\ \text{Quality}(\mathcal{S}_{c_i \to c_i^0}, \mathcal{M}, Q(c_i)). \quad (8)$$

Proof. By the rule (7), the candidate $c_i^*$ is adopted only if $\Delta_i > 0$, where $\Delta_i$ is defined as the change in Quality on $Q(c_i)$ between $\mathcal{S}_{c_i \to c_i^*}$ and $\mathcal{S}_{c_i \to c_i^0}$. Otherwise, the original name is retained and the two schemas are identical on $Q(c_i)$. Either way, inequality (8) holds. □

*Remark 3 (Scope of the Guarantee).* Proposition 1 is *per-column local*: it bounds only the change on $Q(c_i)$ when $c_i$ alone is renamed, and does not extend to a global non-degradation guarantee. LLM-based systems consume the entire schema as prompt context, so renaming one column can affect queries that do not reference $c_i$—via schema-linking, candidate-table pruning, or query-decomposition—effects not bounded by $\Delta_i$; and renaming multiple columns jointly can create schema-level interactions that Phase 3's per-candidate verification does not detect. We therefore characterize EGRefine as *empirically regression-resistant* rather than provably non-degrading: the conservative rule sharply reduces harmful renamings (15/18 configurations net positive $\Delta$, §6.3) but does not eliminate them under cross-column or prompt-level interactions.

*3.4.3 Global Conflict Resolution.* After all per-element selections are made, we perform a global consistency check. If two columns $c_i, c_j$ within the same scope have been assigned identical refined names ($c_i^* = c_j^*$), a *naming collision* arises, violating the admissibility constraint. We resolve such collisions by retaining the refinement with the higher $\Delta$ and reverting the other to its next-best non-conflicting candidate. This process repeats for at most two iterations, after which any remaining conflicts are resolved by retaining the original names.

*3.4.4 Query Equivalence via VIEW Synthesis.* The refined schema $r(\mathcal{S})$ is materialized as a set of SQL `CREATE VIEW` statements aliasing original column names to their refined counterparts:

```
CREATE VIEW t̂ AS SELECT
    c_1 AS r(c_1), ..., c_n AS r(c_n) FROM t,    (9)
```

for each table $t$ with columns $c_1, \ldots, c_n$. Downstream models generate SQL referencing the refined names; the database engine resolves these to underlying columns during query planning, with no intermediate string rewriting.

Proposition 2 (Query Equivalence under DBMS View Semantics). *Let $r$ be an admissible refinement mapping, and let $V = \text{VIEW}(r, \mathcal{S})$ be the corresponding view definitions. For any read-only SQL query $q'$ over $r(\mathcal{S})$ consisting of relational-algebra core operators (projection, selection, equi-join, set operations) over the views, there exists a semantically equivalent query $q$ over $\mathcal{S}$ such that:*

$$\text{exec}(q,\ \mathcal{S}) = \text{exec}(q',\ r(\mathcal{S})). \quad (10)$$

*For SQL features beyond the core algebra (aggregation,* `ORDER BY`*,* `LIMIT`*,* `DISTINCT`*, scalar functions, quoted identifiers,* `NATURAL`

**Algorithm 1** EGRefine Pipeline

**Require:** Schema $\mathcal{S}$, models $\mathcal{M}$, queries $Q$, data samples $D$
**Ensure:** Refined schema $\mathcal{S}^*$ (as SQL VIEWs)

$A \leftarrow$ Phase1-Screen$(\mathcal{S}, \mathcal{M}_{\text{screen}})$ ▷ LLM screening
$A \leftarrow A \setminus$ StructuralExclude$(\mathcal{S})$ ▷ Remove FK / join keys
**for** each column $c_i \in A$ **do**
  Cand$(c_i) \leftarrow$ Phase2-Generate$(c_i, \mathcal{S}, D)$
  $c_i^*, \Delta_i \leftarrow$ Phase3-Verify$(c_i, \text{Cand}(c_i), Q, \mathcal{M})$
**end for**
$R \leftarrow \{(c_i, c_i^*) \mid \Delta_i \geq \tau_{\min}\}$
$R \leftarrow R \cup$ PropagatePK$(R, \mathcal{S})$ ▷ PK→FK
$\mathcal{S}^* \leftarrow$ Phase4-Synthesize$(R, \mathcal{S})$ ▷ CREATE VIEW
**return** $\mathcal{S}^*$

`JOIN`*), equivalence is provided by DBMS view-expansion semantics [19].*

Proof. For the relational-algebra core, view definition (9) implements a rename $\rho$: $\hat{t} = \rho_r(t)$. Since $\rho$ commutes with selection, projection, equi-join, and set operations [1] and $r$ is admissible ($r^{-1}$ well-defined within each scope), any expression $e'$ over $r(\mathcal{S})$ rewrites to an equivalent $e = e'[r^{-1}]$ over $\mathcal{S}$, establishing (10). Constructs outside this core are handled by DBMS view-expansion semantics (a view query is evaluated as if the definition were textually substituted [19]); we verified empirically across Dr.Spider, BIRD, and BEAVER that no execution discrepancy arises between gold SQL on $\mathcal{S}$ and its $r$-aliased counterpart on $V$. □

Together, Observation 1, Theorem 1, Proposition 1 (with Remark 3), and Proposition 2 ground our method: the problem is intractable in general, but the greedy decomposition is locally non-degrading at the column level and the output is non-destructive at the database level—provably for the relational core, empirically beyond it. Section 4 presents EGRefine, a concrete instantiation of this framework.

# 4 METHOD: EGREFINE

EGRefine is a four-phase pipeline that approximately solves Problem 1 via the column-wise decomposition of §3. Algorithm 1 provides a high-level overview.

**Deployment setting.** EGRefine is an offline preprocessing step: it produces a refined schema once and amortizes the cost over future queries. The input includes a representative query workload $Q$ with ground-truth SQL—reflecting deployments where business questions and expert-authored SQL characterize how the schema is used. We use benchmark dev sets as proxies; settings without ground-truth SQL are outside scope (§7). As in workload-driven physical design [2, 6], $Q$ is a representative sample—not the universe of future queries—and the design is re-run on drift; §5.7 shows gains transfer to unseen queries from the same distribution.

## 4.1 Phase 1: LLM-Based Schema Screening

The refinement search space is exponential (Observation 1), so Phase 1 selects a subset $A \subseteq C$ of refinement-improvable columns ($|A| = n \ll m$). We use a lightweight LLM (Qwen3.5-27B) as a schema quality assessor, prompting it with the binary question *"Could this column name cause confusion for a Text-to-SQL model?"* and five contextual signals: database domain description, table name and column data type, neighboring column names, 5 sample rows, and explicit criteria covering abbreviations, domain-specific polysemy, single-letter codes, and generic vocabulary. LLM screening captures semantic ambiguity beyond surface-level rules—e.g., `label` passes lexical checks yet denotes a carcinogenicity indicator in toxicology.

**Structural Exclusion.** A structural filter preserves join integrity: primary keys are allowed (with refined names propagated to FKs in Phase 4); foreign keys are excluded (driven by the corresponding PK); cross-table homonyms sharing both name and data type are excluded as likely implicit join keys. This design favors recall—false positives incur no harm since Phase 3's conservative rule retains their original names.

## 4.2 Phase 2: Context-Aware Candidate Generation

For each candidate column $c_i \in A$, we prompt an LLM to generate $k$ alternative names (default $k$=3) using four context elements: a one-sentence database domain description; neighboring columns with names, types, and 5 sample values; 20 sample values from the target column; and conservative renaming guidelines (include the original name if already clear, avoid over-specification, interpret data values in light of the domain). The LLM returns a ranked JSON list, forming the augmented set $\text{Cand}^+(c_i) = \{c_i^0\} \cup \text{Cand}(c_i)$.

## 4.3 Phase 3: Execution-Grounded Verification

Instead of relying on the LLM's subjective ranking, Phase 3 evaluates each candidate by its downstream effect on Text-to-SQL execution accuracy—in the spirit of self-consistency [49], but using execution accuracy on a verifier query subset as the signal and applying it at the schema-element rather than per-query granularity.

*4.3.1 Query Subset Selection.* For each candidate column $c_i$, we extract the relevant query subset $Q(c_i) = \{(nl, sql^*) \in Q \mid c_i \in \text{cols}(sql^*)\}$. If $Q(c_i) = \emptyset$, execution-based verification is impossible and we retain the original name.

*4.3.2 Per-Candidate Scoring.* For each candidate $c_j' \in \text{Cand}^+(c_i)$, we (1) construct a temporary schema $\mathcal{S}[c_i \to c_j']$ via `CREATE VIEW`; (2) for each $nl \in Q(c_i)$, invoke the Text-to-SQL model on the modified schema; (3) execute predicted SQL against the views and compare with gold SQL on $\mathcal{S}$ (equivalence by Proposition 2). The resulting execution accuracy is:

$$\text{Score}(c_j') = \text{ExAcc}(M, \mathcal{S}[c_i \to c_j'], Q(c_i)). \tag{11}$$

*4.3.3 Multi-Algorithm Aggregation.* To reduce dependence on any single Text-to-SQL system, we aggregate scores:

$$\overline{\text{Score}}(c_j') = \frac{1}{|\mathcal{M}|} \sum_{M \in \mathcal{M}} \text{Score}_M(c_j'). \tag{12}$$

We use $\mathcal{M}$ = {C3, DIN-SQL}, holding out MAC-SQL to test cross-algorithm transfer (§5.4). $|\mathcal{M}|$=2 is empirically motivated (§5.3): $M$=1 overfits and degrades other algorithms, while $M$=2 forces consensus that generalizes.

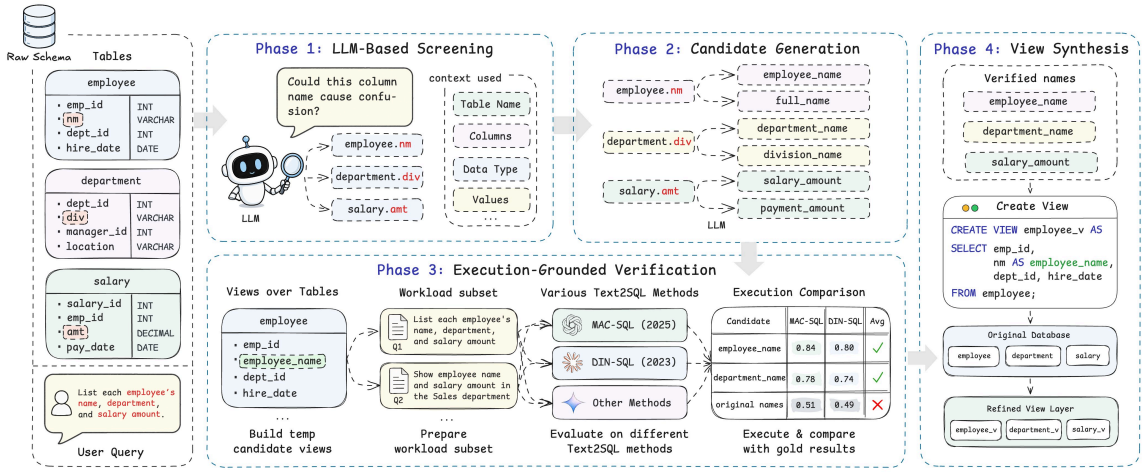


**Figure 1: Overview of the EGRefine pipeline. Four phases turn a raw schema $\mathcal{S}$ (left) into a refined, non-destructive view layer $\mathcal{S}'$ (right): column screening, candidate generation, execution-grounded verification, and VIEW synthesis (§4.1–4.4). The user workload is consumed only by Phase 3, concentrating supervision at a single replaceable bottleneck; the original database is left untouched.**

*4.3.4 Conservative Selection.* The best candidate is selected as:

$$c_i^* = \arg \max_{c'_j \in \text{Cand}^+(c_i)} \overline{\text{Score}}(c'_j), \tag{13}$$

subject to:

$$\Delta_i = \overline{\text{Score}}(c_i^*) - \overline{\text{Score}}(c_i^0) \geq \tau_{\min}, \tag{14}$$

where $\tau_{\min}$ is a minimum improvement threshold; if $\Delta_i < \tau_{\min}$, the original name is retained. This filters marginal noise and instantiates Proposition 1; Algorithm 2 details the procedure.

*4.3.5 Computational Cost.* Phase 3 dominates the offline cost: $O(|A| \cdot k \cdot |\mathcal{M}| \cdot \overline{|Q(c)|})$ Text-to-SQL inferences. On Dr.Spider-Abbr (Qwen3.5-27B) with $|A|$=665, $k$=3, $|\mathcal{M}|$=2, $\overline{|Q(c)|}\approx$10, this yields $\approx 4 \times 10^4$ inferences (≈4–6 h on one A800 at 16-parallel concurrency); on BIRD, $\approx 9 \times 10^3$ inferences (≈1–2 h). This one-time cost amortizes across future queries and multiple downstream models (§5.4). For very large schemas, Phase 1 reduces $|A|/|C|$ to under 35% (Table 8).

## 4.4 Phase 4: Non-Destructive Schema Synthesis

The final phase materializes the refinement as SQL `CREATE VIEW` statements (Eq. 9), the non-destructive layer guaranteed by Proposition 2: original tables and data are preserved verbatim and remain queryable, while downstream systems query the refined views directly (no data duplication).

**Engineering scope.** `CREATE VIEW` targets read-only Text-to-SQL workloads (BI queries, dashboards, ad-hoc analytics). The view layer uses only standard `CREATE VIEW` column aliasing, so the equivalence of Proposition 2 is portable across standard SQL engines. Production deployments may still warrant routine dialect handling (reserved-keyword quoting, identifier casing, view updateability, optimizer interaction, access control)—deterministic engineering rather than threats to read-only correctness.

**Algorithm 2** Execution-Grounded Verification (Phase 3)

**Require:** Column $c_i$, candidates $\text{Cand}^+(c_i)$, queries $Q(c_i)$, models $\mathcal{M}$, data samples $D_s$
**Ensure:** Selected name $c_i^*$, improvement $\Delta_i$

```
if Q(c_i) = ∅ then
    return c_i^0, skipped        ▷ retain original: no queries to verify
end if
for each c'_j ∈ Cand+(c_i) do
    Score(c'_j) ← 0
    for each M ∈ M do
        for each (nl, sql*) ∈ Q(c_i) do
            ŝql ← M(nl, S[c_i → c'_j], D_s)
            Execute ŝql on view S[c_i → c'_j]; compare with exec(sql*) on S
        end for
        Score(c'_j) += ExAcc/|M|
    end for
end for
c_i* ← arg max_{c'_j} Score(c'_j)
Δ_i ← Score(c_i*) − Score(c_i^0)
if Δ_i < τ_min then
    return c_i^0, 0              ▷ conservative: keep original
end if
return c_i*, Δ_i
```

*4.4.1 PK→FK Name Propagation.* When a PK column is renamed, all FKs referencing it must follow to maintain join consistency. For each renamed PK $(T_p, c_p) \to c'_p$, we apply the same rename to every FK column referencing $(T_p, c_p)$. This propagation is deterministic and requires no LLM calls or execution verification. Columns with

$Q(c_i) = \emptyset$ retain their original names. The complete set of view definitions constitutes the refined schema $\mathcal{S}^*$.

# 5 EXPERIMENTS

We organize our evaluation around four research questions: **RQ1**: Can EGRefine recover performance lost to schema naming degradation? **RQ2**: How much does each pipeline component contribute, and how sensitive is the method to its hyperparameters? **RQ3**: Does refinement transfer across model families (*refine-once, serve-many-models*)? **RQ4**: Is EGRefine effective on real-world schemas? Four supporting analyses (§5.6–§5.9) and a mechanism dissection (§6) follow.

## 5.1 Experimental Setup

*5.1.1 Benchmarks.* **Dr.Spider** [4] serves as our primary benchmark with two schema perturbation subsets: *Schema-Abbreviation* (Dr.Spider-Abbr; 691 perturbed columns across 90 databases) systematically replaces column names with abbreviations (e.g., `country` → `cntry`), and *Column-Synonym* replaces them with semantic equivalents. Built on Spider [54] (1,034 dev queries expanded to 2,853 by perturbation), Dr.Spider provides a controlled setting to measure *recovery capability*: how much of the performance lost to schema degradation can be restored. Unless otherwise noted, "Dr.Spider" refers to the Schema-Abbreviation subset.

**BIRD** [26] (1,534 queries, 11 databases) validates EGRefine on naturally occurring schemas with domain-specific abbreviations and irregular naming. We deliberately *exclude all hints* to simulate realistic deployment conditions where human-curated annotations are unavailable.

**BEAVER** [10] (88 queries, 4 databases, 1,439 columns) provides enterprise-scale validation with complex schemas from real data warehouses.

*5.1.2 Text-to-SQL Systems.* We evaluate three representative algorithms covering distinct prompting paradigms: C3 [13] (zero-shot), DIN-SQL [33] (decomposed prompting), and a MAC-SQL-style [47] multi-agent system (Selector → Decomposer → Refiner). We use a unified zero-shot prompting setup across all three for evaluation consistency; reported numbers reflect baselines under this harness, not original published numbers. The default backbone is Qwen3.5-27B; cross-model analysis (RQ3) additionally evaluates Qwen3.5-9B, Gemma3-27B, and Phi-4-14B.

*5.1.3 Refinement Baselines.* **No Refinement** (NoRef): the (possibly degraded) schema is used as-is. **LLM-Direct**: Phases 1–2 of EGRefine, then select the LLM's top-ranked candidate without execution verification (isolates Phase 3's contribution). **SNAILS** [29]: its released naturalness classifier screens unnatural columns, which are then renamed by Qwen3.5-27B and accepted without execution verification (§2.2). **Description Annotation**: LLM-generated column descriptions are injected as schema-prompt comments while identifiers remain unchanged (used in §5.8 only).

*5.1.4 Metrics.* We report Execution Accuracy (ExAcc). For Dr.Spider, we additionally report *Recovery Rate*:

$$\text{RecRate} = \frac{\text{ExAcc}_{\text{refined}} - \text{ExAcc}_{\text{degraded}}}{\text{ExAcc}_{\text{clean}} - \text{ExAcc}_{\text{degraded}}} \times 100\% \qquad (15)$$

**Table 1: Execution Accuracy (%) on Dr.Spider Schema-Abbreviation (2,853 queries, Qwen3.5-27B). Recovery Rate per Eq. 15. †SNAILS [29] uses its released naturalness classifier to screen columns, which are then renamed by Qwen3.5-27B and accepted without execution verification, isolating the selection signal (§2.2).**

| Method | C3 | DIN-SQL | MAC-SQL |
|---|---|---|---|
| Clean (Original Spider) | 72.77 | 80.41 | 63.79 |
| Degraded (Abbreviated) | 70.87 | 76.73 | 59.59 |
| + LLM-Direct | 72.34 | 75.85 | 57.17 |
| + SNAILS† | 67.33 | 73.33 | 58.15 |
| + **EGRefine** | **73.43** | **79.21** | **60.22** |
| Δ (EGRefine − Degraded) | +2.56 | +2.48 | +0.63 |
| Recovery Rate | 134.7% | 67.4% | 15.0% |

where $\text{ExAcc}_{\text{clean}}$ is accuracy on the unperturbed Spider schema; values above 100% indicate the refined schema exceeds the original.

*5.1.5 Implementation.* All EGRefine phases use Qwen3.5-27B deployed locally via vLLM (zero API cost), with $k$=3 candidates per column, $N_s$=20 sampled rows, $\tau_{\min}$=0.05, and verifier set $\mathcal{M}$ = {C3, DIN-SQL}. The refinement backbone never acts as its own downstream evaluator: downstream systems treat the refined schema as opaque input. Reported gains on C3 and DIN-SQL (the verifier set) are *in-loop* measurements; gains on *MAC-SQL* are *out-of-loop*, reflecting refinements selected without reference to its behavior.

## 5.2 RQ1: Recovery from Schema Degradation

Table 1 presents the main results on Dr.Spider Schema-Abbreviation. Schema abbreviation degrades ExAcc by 1.9–4.2 percentage points (pp); EGRefine recovers this loss across all three systems, matching or exceeding the clean baseline on C3 (134.7% recovery, indicating refinement also improves over Spider's original names) and recovering 67.4% / 15.0% on DIN-SQL / MAC-SQL respectively. The unverified baselines, in contrast, inject harmful noise: SNAILS's naturalness-driven renaming *degrades all three systems below the unrefined schema* (−3.54/−3.40/−1.44 pp; two-sided McNemar $p$<0.05), and LLM-Direct worsens DIN-SQL (−0.88 pp) and MAC-SQL (−2.42 pp). EGRefine outperforms SNAILS by 6.1/5.9/2.1 pp ($p$<0.001), isolating execution verification—not naming naturalness—as the source of its gains. On Column-Synonym (C3 only), EGRefine achieves +2.25 pp recovery (64.87% → 67.12%), generalizing to non-abbreviation degradations.

## 5.3 RQ2: Component Ablation and Sensitivity

Table 2 isolates the contribution of each component on Dr.Spider Schema-Abbreviation.

**w/o Execution** (= LLM-Direct) removes Phase 3 verification and causes the largest degradation across all three systems—on Dr.Spider-Abbr it leaves DIN-SQL and MAC-SQL *below* their no-refinement baselines (−0.88 and −2.42 pp), meaning naive LLM-based refinement actively harms downstream accuracy. The query-level flip analysis in §6.3 confirms this pattern.

**Table 2: Ablation on Dr.Spider Schema-Abbreviation (ExAcc %). $M$=1 uses C3-only verification; full pipeline uses $M$=2 (C3+DIN-SQL).**

| Variant | C3 | DIN-SQL | MAC-SQL |
|---|---|---|---|
| **EGRefine (full, $M$=2)** | **73.43** | **79.21** | **60.22** |
| w/o Execution | 72.34 | 75.85 | 57.17 |
| w/o Conservative | 73.01 | 78.83 | 58.92 |
| w/o Screening | 71.08 | 77.32 | 58.78 |
| w/o Multi-Algo ($M$=1) | 73.36 | 76.34 | 59.48 |
| Degraded (no refine) | 70.87 | 76.73 | 59.59 |

**w/o Conservative Rule** removes the $\tau_{\min}$ threshold, applying all renamings regardless of verification score. The drop (−0.42/−0.38/−1.30) is much smaller than under **w/o Execution** (−1.09/−3.36/−3.05): execution verification is the dominant mechanism, with abstention as secondary refinement.

**w/o Phase 1 Screening** sends all non-structural columns to Phase 2. The second-largest drop (−2.35/−1.89 on C3/DIN) shows that indiscriminate candidate generation dilutes Phase 3's signal.

**w/o Multi-Algo Verification** ($M$=1, C3 only) exposes verifier overfitting: while C3 itself gains comparably (+2.49 vs +2.56 with $M$=2), the refined schema *degrades* DIN-SQL (−0.39 pp) and slightly hurts MAC-SQL (−0.11 pp). This mechanistically explains why held-out cross-algorithm transfer (§5.4) works: $M$=2 forces consensus refinements that generalize to unseen algorithms.

**Sensitivity to the conservative threshold $\tau_{\min}$.** Sweeping $\tau_{\min} \in \{0.01, 0.03, 0.05, 0.10\}$ (committing 104/89/81/54 columns) gives C3/DIN/MAC Δ vs. NoRef of +0.70/−0.25/−0.81, +0.81/+1.15/−0.11, +2.56/+2.48/+0.63, and +0.70/+0.94/+0.56 respectively. *Too permissive* (0.01) over-commits and causes net regressions on DIN-SQL/MAC-SQL, validating $\tau_{\min}>0$; *too strict* (0.10) shrinks gains to roughly one-third of the optimum. The chosen $\tau_{\min}$=0.05 Pareto-dominates—highest ExAcc on every algorithm, worst-case spread 2.7 pp—so the pipeline is not finely tuned to one threshold. The sweep also reinforces §6.4: MAC-SQL's Δ increases monotonically in $\tau_{\min}$ over {0.01, 0.03, 0.05}, indicating heightened sensitivity to low-confidence renamings.

**Sensitivity to the verifier-set choice.** We test whether reported gains depend on cherry-picking a favorable verifier set by re-running the pipeline with an alternative $\mathcal{M}'$={DIN, MAC}; C3 now serves as the held-out evaluator (Table 3). EGRefine yields positive gains on *every* algorithm under both configurations ($\Delta \in [+0.63, +4.47]$). The held-out role does not determine performance: C3—now held out—achieves the strongest result (+4.47 pp, exceeding the clean Spider baseline by 2.57 pp), ruling out "held-out exclusion" as the cause of MAC-SQL's modest gain in the original configuration. MAC-SQL's gain stays bounded even when it is itself a verifier, a pattern §6.4 traces to its agent design.

## 5.4 RQ3: Cross-Model Transferability

A practical question is whether a refined schema produced by one model benefits a different model at serving time, enabling *refine-once, serve-many-models* deployment.

**Table 3: Cross-verifier robustness on Dr.Spider Schema-Abbreviation (Qwen3.5-27B). Original $\mathcal{M}$ = {C3, DIN}; alternative $\mathcal{M}'$ = {DIN, MAC}. Held-out role is shaded.**

| Verifier set | Algo | ExAcc (%) | Δ | Recovery |
|---|---|---|---|---|
| {C3, DIN} (original) | C3 | 73.43 | +2.56 | 134.7% |
| | DIN-SQL | 79.21 | +2.48 | 67.4% |
| | MAC-SQL | 60.22 | +0.63 | 15.0% |
| {DIN, MAC} (alternative) | C3 | **75.34** | **+4.47** | **235.3%** |
| | DIN-SQL | 79.87 | +3.14 | 85.2% |
| | MAC-SQL | 63.17 | +3.58 | 85.2% |

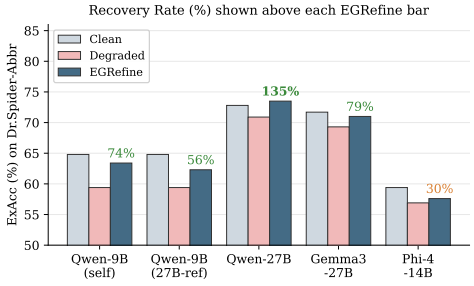


**Figure 2: Cross-model results on Dr.Spider Schema-Abbreviation (DIN-SQL algorithm). All EGRefine bars use 27B-refined schema unless marked "self". Recovery Rate (%) annotated above each EGRefine bar.**

*5.4.1 Multi-Backbone Evaluation.* Figure 2 reports results across four model families on Dr.Spider Schema-Abbreviation, all using the same 27B-refined schema (except "self-refine" rows). EGRefine delivers positive recovery across all families, with DIN-SQL recovery exceeding 100% on Gemma3-27B (133.7%) and Phi-4 (188.1%)—refined schemas can unlock cross-family gains beyond the original clean baseline.

*5.4.2 Transfer Matrix.* Table 4 presents the full refine×eval transfer matrix for Qwen3.5-9B and 27B. **Strong→Weak transfer is nearly lossless**: 27B-refined schema evaluated on 9B trails 9B self-refinement by less than 1 pp on every system. **Weak→Strong transfer can match or exceed self-refinement**: 9B-refined schema evaluated on 27B *surpasses* 27B self-refinement on DIN-SQL (79.53 vs. 79.21). **Weak models should not self-refine**: Phi-4's self-refined schema yields −0.63 pp on C3 versus +0.74 pp under 27B refinement—weak backbones lack the capability to generate high-quality candidates, making Phase 2 the bottleneck.

## 5.5 RQ4: Deployment on Real-World Schemas

While Dr.Spider uses controlled perturbations, real-world schemas exhibit naturally occurring naming issues. We validate on BIRD (curated academic schemas) and BEAVER (enterprise data warehouses).

*5.5.1 BIRD: Execution Verification is Essential.* On BIRD's professionally curated schemas, EGRefine conservatively refines only 28 of 798 columns (3.5%), yielding modest but positive aggregate

**Table 4: Refine × Eval transfer matrix on Dr.Spider Schema-Abbreviation. Δ = improvement over eval model's no-refinement baseline.**

| Refine | Eval | C3 (Δ) | DIN (Δ) | MAC (Δ) |
|---|---|---|---|---|
| 9B | 9B | 63.34 (+3.96) | 75.81 (+5.18) | 53.63 (+2.11) |
| 27B | 9B | 62.36 (+2.98) | 75.60 (+4.97) | 53.49 (+1.97) |
| 9B | 27B | 73.22 (+2.35) | **79.53** (+2.80) | 59.34 (−0.25) |
| 27B | 27B | **73.43** (+2.56) | 79.21 (+2.48) | **60.22** (+0.63) |

**Table 5: Execution verification necessity on BIRD (Qwen3.5-9B, 1,534 queries, no hints). LLM-Direct = verification-free LLM baseline.**

| Method | C3 | DIN-SQL | MAC-SQL |
|---|---|---|---|
| No Refinement | 28.75 | 31.75 | 20.60 |
| + LLM-Direct | 28.23 ↓0.52 | 30.12 ↓1.63 | 19.17 ↓1.43 |
| + **EGRefine** | **30.25** ↑1.50 | **33.25** ↑1.50 | 20.14 ↓0.46 |
| Separation (EGR−Dir) | +2.02 | +3.13 | +0.97 |

improvements on 27B (+0.20/+0.65/+1.24 on C3/DIN/MAC). A finer-grained analysis in §6.6 reveals an algorithm-specific differential: on the 126-query touching subset, DIN-SQL gains +6.35 pp while C3 (−1.59 pp) and MAC-SQL (−3.17 pp) do not benefit—a pattern we trace to agent-level brittleness (§6.4).

**Robustness of the BIRD aggregate effect.** Paired bootstrap 95% CIs show the per-cell BIRD gains are not individually significant (e.g., 27B DIN-SQL +0.65, CI $[-0.5, +1.8]$; strongest 9B DIN-SQL +1.50, CI $[+0.07, +2.9]$)—the expected consequence of BIRD's low refinement coverage (3.5% of columns, §6.1), not a null effect. We therefore rest the claim on convergent evidence: gains are positive in 6/7 cells, and four independent analyses below (workload-holdout §5.7, touching-subset §6.6, 9B reproduction Table 5, coverage–improvement linearity §6.1) agree in direction and magnitude. The controlled Dr.Spider gains, at far higher coverage, are by contrast individually significant for C3 and DIN-SQL (§6.3).

The critical finding emerges on the weaker 9B backbone (Table 5): LLM-Direct *degrades* all three systems (−0.52/−1.63/−1.43), while EGRefine improves C3 and DIN-SQL (+1.50/+1.50) with marginal regression on MAC-SQL (−0.46). The 0.97–3.13 pp separation between verified and unverified refinement establishes execution-grounded verification as necessary for safe refinement on smaller models.

*5.5.2 BEAVER: Where the Signal Runs Out.* BEAVER's enterprise schemas (1,439 columns across 4 databases, 88 queries) present an unusual regime: baseline ExAcc is substantially lower than any other setting we examined (C3: 7.95%, DIN-SQL: 7.95%, MAC-SQL: 3.41%) due to schemas whose complexity, domain specificity, and scale exceed current Text-to-SQL system capabilities. This regime exposes a fundamental precondition: execution-grounded verification requires a non-trivial baseline for discriminative signal.

**Qwen3.5-27B abstains; MiniMax-M2.7 commits but does not move downstream ExAcc.** With Qwen3.5-27B as refiner, Phase 3 rejects all 106 Phase 1 candidates—the conservative rule operating as designed when Δ scores cannot be reliably distinguished from noise. Testing whether refiner capability is the bottleneck, we repeat with MiniMax-M2.7: Phase 1 screens 172 candidates (vs. Qwen's 106) and Phase 3 commits 4 refinements (vs. 0). Despite this, downstream ExAcc on C3 and DIN-SQL is unchanged (both 7.95%, 7/88; +0.00 pp): Phase 3 in fact screened and execution-scored the columns these queries reference (e.g., `group.extra`, `user.extra`), yet no candidate rename changed ExAcc (Δ=0), so the conservative rule kept them. On BEAVER's hardest schemas, naming is simply not the accuracy bottleneck for the outcome-determining columns—the invariance is a downstream ceiling, not a verification failure (cf. §6.5). The lone exception is MAC-SQL (3.41%→5.68%, 3/88→5/88; +2.27 pp), which inverts the Dr.Spider pattern where MAC benefits least: on BEAVER's extremely difficult queries, MAC-SQL's multi-agent decomposition rarely reaches the aggressive re-decomposition stage that causes regressions on easier benchmarks, providing independent out-of-distribution validation of the agent-instability hypothesis (§6.4).

**Table 6: BIRD evidence × refinement ablation (Qwen3.5-27B, 1,534 queries). Evidence and EGRefine are complementary.**

| Configuration | C3 | DIN | MAC |
|---|---|---|---|
| No Refinement | 41.53 | 38.59 | 29.99 |
| + Evidence | 54.24 | 62.13 | 46.74 |
| + EGRefine | 41.72 | 39.24 | 31.23 |
| + Both | **54.82** | **63.56** | **47.33** |
| Δ (Both − Evidence) | +0.58 | +1.43 | +0.59 |

## 5.6 Complementarity with Domain Knowledge

BIRD provides optional per-query *evidence* (domain hints). A 2×2 ablation (Table 6) confirms EGRefine provides additional gains on top of evidence across all three systems on 27B (+0.58/+1.43/+0.59): structural disambiguation (column renaming) and semantic disambiguation (domain hints) are orthogonal—evidence explains a column's meaning in a specific query context, while refinement makes the column name inherently more interpretable regardless of query.

## 5.7 Workload-Holdout Validation

A natural question is whether BIRD gains reflect overfitting to the queries used during Phase 3 verification, since our setup uses BIRD's dev set as both the representative workload $Q$ and the evaluation set. We focus the holdout on BIRD because its queries were curated alongside the schemas, making coupling more acute than on Dr.Spider (where queries predate any perturbation; additionally, Dr.Spider's small academic schemas lack the column-description metadata required for our isolation protocol).

We constructed an independent holdout against a fixed EGRefine refined schema (unchanged from main experiments). We generated 110 (NL, SQL) pairs for BIRD's 11 databases (10 per database) using DeepSeek-V4-Pro under a strict isolation protocol: NL questions were authored using only BIRD's official column descriptions, with

**Table 7: Identifier replacement vs. description annotation (C3 + Qwen3.5-27B, $n$=2853). Description annotations are LLM-generated SQL comments. ExAcc differs slightly from Table 1 due to prompt-formatting tokenization effects.**

| Schema | Description | ExAcc | Δ vs. NoRef |
|---|---|---|---|
| NoRef | no | 70.83 | — |
| NoRef | yes | 72.48 | +1.65 |
| EGRefine | no | 73.46 | +2.63 |
| EGRefine | yes | 74.69 | **+3.86** |

sample-data headers replaced by `col_1`, `col_2`, …; only after the NL was finalized did the model see real column names to author gold SQL. Execution validation retained 96 queries (executed successfully, non-empty result sets ≤1000 rows, non-duplicate within database). These are non-trivial: 65/96 aggregations, 37/96 `GROUP BY`, 32/96 `ORDER BY`, 22/96 subqueries; mean SQL length 184 characters, mean JOIN count 0.93.

On these 96 unseen queries the refined schema scores 48.96 vs. 47.92 NoRef (+1.04 pp; 95% CI [−3.1, +5.2], McNemar $p$=1.0), against +0.19 pp on the 1,534 in-loop dev queries—directionally larger out-of-loop, though $n$=96 is far too small for significance. We treat this as preliminary evidence mitigating the workload-overfit concern rather than decisive refutation (single cell, $n$=96, C3 alone), consistent with our pipeline structure: Phases 1–2 produce candidates without consulting queries, and Phase 3's discrete execution-grounded signal lacks the bandwidth to encode dev-set-specific patterns into column names.

## 5.8 Identifier Replacement vs. Description Annotation

A natural alternative to renaming columns is to leave column identifiers untouched and inject column descriptions as SQL comments into the schema prompt. We test whether this non-invasive route is sufficient.

**Protocol.** We reuse EGRefine's Phase 1 screened column set on Dr.Spider-Abbr (665 columns) and prompt Qwen3.5-27B to generate one 8–25-word SQL comment per column from its name, neighbor columns, and 20 sampled values. The descriptions are injected as inline SQL comments (e.g., `gf TEXT, -- The country's form of government, e.g., Republic, Monarchy`); column identifiers are unchanged. Critically, the description baseline performs no execution verification—a single-pass annotation matching the simplest deployment without our full pipeline. We evaluate four cells with C3 + Qwen3.5-27B (Table 7).

**Result.** Description annotation alone yields +1.65 pp—non-trivially helpful, ruling out a strawman baseline. Identifier replacement (EGRefine) yields +2.63 pp, exceeding description annotation by 0.98 pp on the same Phase 1 column set and same backbone. The combined treatment yields +3.86 pp, 0.42 pp below the linear sum of the two—indicating partial redundancy: once a column is renamed to a semantically clear identifier, an additional description provides diminishing returns.

**Scope of the comparison.** Two design differences confound a strict "identifier vs. description" attribution: EGRefine adds execution-grounded selection (Phase 3) absent from the single-pass description baseline, and the descriptions are LLM-generated rather than expert-authored. We therefore frame the conclusion narrowly—under the tested configuration, identifier replacement outperforms by 1.6× and the two mechanisms remain partially complementary; a stronger description baseline with Phase 3-style selection is left to future work.

## 5.9 Phase 1 Screening Funnel

Table 8 traces the search-space reduction from raw columns to final refinements. Across all benchmarks, ∼66–93% of columns are filtered by structural exclusion and LLM screening (Phase 1), and Phase 3 verification further prunes to 3.5–7.6% of the original schema. On refined columns, the average quality gain ($\overline{\Delta}_{\text{refined}}$) ranges from 19.95 to 26.62 pp—each committed refinement delivers substantial benefit on its affected query subset. On BEAVER, Qwen3.5-27B's conservative Phase 3 commits zero; MiniMax-M2.7 raises Phase 1 recall to 12.0% and commits 4 refinements, yet downstream impact remains bounded by the benchmark's column-insensitive query structure (§6.5).

# 6 ANALYSIS AND INSIGHTS

Section 5 established that EGRefine delivers consistent improvements; this section dissects *why* it works and *where its limits lie*, through six analyses spanning the database (§6.1), column (§6.2), and query (§6.3) levels, plus three diagnostic studies (§6.4–§6.6).

## 6.1 Database-Level: Coverage Predicts Improvement

A central question is whether aggregate ExAcc improvement tracks the amount of schema actually modified. We analyze 606 (benchmark, backbone, algorithm, database) configurations across Dr.Spider-Abbr and BIRD, defining *coverage* as $n_r^{(D)}/m^{(D)}$ for each database.

Across the 273 database-algorithm pairs with zero refined columns, Δ is identically zero (the refined schema is byte-identical to the original), confirming the conservative rule's safety property: when Phase 3 finds no improvement, the schema is left untouched.

Aggregating by (benchmark, backbone, algorithm) and weighting by per-database query count, coverage predicts improvement cleanly: each percentage point of coverage yields approximately +0.25 to +0.40 pp of benchmark-wide ExAcc gain. Binning by coverage (Figure 3) confirms this: databases with >15% coverage achieve a +6.30 pp $n$-weighted aggregate gain with 62.5% of configurations exhibiting positive deltas. This explains the BIRD results: with 3.39% query-weighted coverage (3.51% column-fraction, §5.9), the observed +0.20 to +1.24 pp falls within the predicted +0.8 to +1.4 pp range. The modest BIRD improvement is not weak methodology but a direct consequence of BIRD's schemas already being well-named—only 3.5% of columns meet the refinement bar.

## 6.2 Column-Level: LLM Top-1 is Unreliable

RQ2's ablation showed that removing Phase 3 degrades performance. We now measure *how often* Phase 3 overrides the LLM's top-ranked candidate (Figure 4).

**Table 8: Phase 1 screening funnel. $m$: total columns, $n$: candidates after Phase 1, $n_r$: finally refined after Phase 3. $\overline{\Delta}_{\text{refined}}$ is averaged over refined columns on their affected query subsets.**

| Benchmark | Backbone | DBs | $m$ | $n$ (P1) | $n_r$ (final) | Excl. rate | Compr. $n_r/m$ | $\overline{\Delta}_{\text{refined}}$ |
|---|---|---|---|---|---|---|---|---|
| Dr.Spider-Abbr | Qwen3.5-27B | 90 | 1985 | 665 | 81 | 66.5% | 4.08% | 20.68 pp |
| Dr.Spider-Abbr | Qwen3.5-9B | 90 | 1985 | 841 | 151 | 57.6% | 7.61% | 20.94 pp |
| BIRD | Qwen3.5-27B | 11 | 798 | 251 | 28 | 68.5% | 3.51% | 26.62 pp |
| BIRD (+evidence) | Qwen3.5-27B | 11 | 798 | 252 | 28 | 68.4% | 3.51% | 21.76 pp |
| BIRD | Qwen3.5-9B | 11 | 798 | 308 | 39 | 61.4% | 4.89% | 19.95 pp |
| BEAVER | Qwen3.5-27B | 4 | 1439 | 106 | 0 | 92.6% | 0.00% | — |
| BEAVER | MiniMax-M2.7 | 4 | 1439 | 172 | 4 | 88.0% | 0.28% | — |

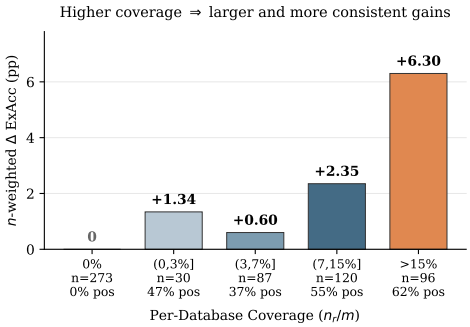


**Figure 3: ExAcc Δ by per-database coverage bin (606 configurations, $n$-weighted by query count). Positive-Δ rate annotated below each bar. Databases with > 15% coverage achieve the largest gains (+6.30 pp) and highest positive rate (62.5%).**

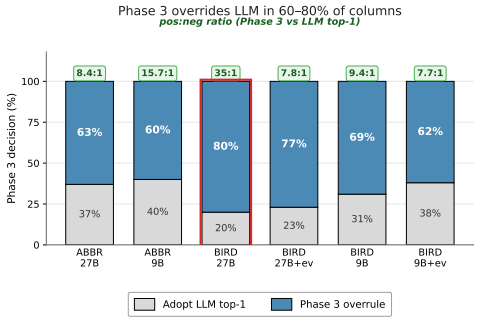


**Figure 4: Phase 3 overrides LLM's top-1 candidate in 60–80% of verified columns. Green text shows the pos:neg ratio when Phase 3 disagrees with LLM (almost always Phase 3 wins). BIRD 27B (highlighted) has the highest override rate (80.5%) and cleanest pos:neg (35:1).**

Across all six configurations, Phase 3 disagrees with the LLM's top choice in 60–80% of cases; on BIRD with 27B, the adoption rate is only 19.5%—four out of five LLM recommendations are overridden. When Phase 3 selects a non-top-1 outcome, the positive-to-negative ratio (Phase 3's pick beats LLM top-1) ranges from ≈8:1 (Dr.Spider-Abbr 27B) to ≈35:1 (BIRD 27B): when Phase 3 disagrees, it is overwhelmingly right.

The dominant overrule category is *overrule-to-original*: LLM proposes a change, but Phase 3 rejects all candidates and retains the original name (62.7% of BIRD-27B verifications, all 106 BEAVER decisions). LLMs systematically over-recommend changes; execution grounding filters this bias.

## 6.3 Query-Level: Flips Reveal Systematic Repair

The finest-grained evidence comes from per-query correctness changes. For each query, we classify its (NoRef → Refined) outcome into four categories and focus on **C→W** (NoRef correct, refined wrong—a regression) and **W→C** (NoRef wrong, refined correct—a genuine repair).

**EGRefine achieves W→C ≥ C→W in 15/18 configurations.** On Dr.Spider-Abbr with 27B, the repair-to-break ratio is 4.04:1 for C3 (97 repairs vs 24 breaks) and 6.46:1 for DIN-SQL (84 vs 13), both significant (two-sided McNemar $p<0.001$), while MAC-SQL is 1.37:1 (67 vs 49; $p=0.11$, n.s.); on 9B DIN-SQL it is 4.15:1 (195 vs 47; $p<0.001$). The three configurations with ratio <1 are all on MAC-SQL with small margins (≤13 queries)—a pattern we trace to its source in §6.4.

**LLM-Direct fails the flip test on 9B BIRD,** achieving ratios of 0.69 on DIN-SQL (56 repairs vs. 81 breaks) and 0.68 on MAC-SQL (47 vs. 69)—it breaks more queries than it repairs. Comparing EGRefine to LLM-Direct on the same (benchmark, backbone, algorithm) cell, EGRefine breaks fewer queries in all 12 cells (median reduction: 63 queries/cell). Phase 3's dominant contribution is preventing regressions, not producing additional repairs.

## 6.4 Error Modes: Why MAC-SQL Benefits Least

Under the main configuration ($\mathcal{M}$ = {C3, DIN-SQL}), MAC-SQL shows the smallest gain from refinement (+0.63 pp on Dr.Spider-Abbr, vs. +2.56 and +2.48 for C3 and DIN-SQL). The cross-verifier experiment (§5.3) confirms this gap is bounded rather than incidental: even when MAC-SQL is itself a verifier and refinements are explicitly tuned to its preferences, its recovery rate (85.2%) matches DIN-SQL's rather than approaching the higher recoveries observed for algorithms with more stable schema linking. We diagnose the cause on the 633 queries referencing refined columns, ruling out two candidate explanations before identifying the true mechanism.

**It is not a usage problem.** MAC-SQL references refined names in 86.1% of its predictions, comparable to C3 (91.3%) and DIN-SQL (92.4%); the 5–6 pp gap cannot explain a 2–4 pp ExAcc gap.

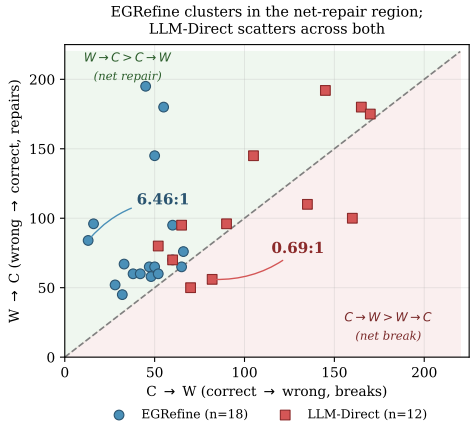


**Figure 5: Query-level C→W vs W→C flips across 30 configurations: 18 EGRefine (incl. 6 BIRD+evidence variants) and 12 LLM-Direct (verification-free baseline). EGRefine clusters in the upper-left net-repair region (15/18 with ratio >1, max 6.46:1); LLM-Direct scatters and includes severe failures below the diagonal (worst DIN-SQL point 0.69:1, worst overall 0.68:1 on MAC-SQL).**

**It is not a dilution problem.** Even on the 633-query touching subset, MAC-SQL gains only +2.84 pp while C3 and DIN-SQL gain +11.53 and +11.22 pp respectively. The small gain is intrinsic, not averaged away.

**It is a structural rewriting problem.** MAC-SQL's Pass→Fail rate on the touching subset is 7.7%—2–4× that of C3 (3.8%) and DIN-SQL (2.1%). Among its 49 regressions, 73% involve structural query edits: 33% change SELECT column count, 24% drop JOIN clauses, 26% drop aggregates, 10% drop `WHERE` clauses. For C3 and DIN-SQL, 77–79% of regressions are non-structural (identical query shape with minor semantic drift). For example, on *"how many types of government are in Africa"* with `gf`→`government_form`, MAC-SQL's refiner re-decomposes the query and drops the `COUNT(DISTINCT)` aggregate. On the weaker 9B backbone MAC-SQL instead improves +2.11 pp, suggesting over-aggressive rewriting is capability-emergent.

This exposes an *agent-level instability* in multi-agent Text-to-SQL systems that is orthogonal to schema linking: clearer column names can prompt over-aggressive query rewriting.

## 6.5 Benchmark-Level Scope: Why BEAVER Probes a Different Task

Is BEAVER's near-zero improvement an EGRefine limitation or a property of the benchmark? Its gold SQL suggests the latter—a problem structurally distinct from standard Text-to-SQL.

**Structural divergence.** BEAVER's gold SQL dwarfs standard benchmarks: median query length 3,312 characters (max 10,168) versus ~115–150 (max ≤1,000) for BIRD/Dr.Spider-Abbr, and median JOIN count 4 (max 13) versus 0–2 (max ≤5)—a 20–40× structural gap. The gold SQL is predominantly ORM-generated (SQLAlchemy-style), featuring `SELECT *` with column aliases over stacked `LEFT OUTER JOIN`s.

**Table 9: Touching-subset decomposition. $\Delta_{\text{touching}}$: per-query effect on queries referencing ≥1 refined column; $\Delta_{\text{full}}$: aggregate effect. Cross-model row: 27B-refined schema served to 9B (DIN-SQL).**

| Cell | Algo | $n_{\text{tch}}$ | $\Delta_{\text{tch}}$ | $\Delta_{\text{full}}$ | Ratio |
|---|---|---|---|---|---|
| Dr.Spider 27B | C3 | 633 | +11.53 | +2.56 | 4.50× |
| Dr.Spider 27B | DIN | 633 | +11.22 | +2.48 | 4.52× |
| Dr.Spider 27B | MAC | 633 | +2.84 | +0.63 | 4.51× |
| Dr.Spider 9B | C3 | 1126 | +10.04 | +3.96 | 2.53× |
| Dr.Spider 9B | DIN | 1126 | +13.14 | +5.19 | 2.53× |
| Dr.Spider 9B | MAC | 1126 | +5.33 | +2.10 | 2.54× |
| BIRD 27B | DIN | 126 | +6.35 | +0.65 | 9.77× |
| BIRD 27B | C3 | 126 | −1.59 | +0.20 | — |
| BIRD 27B | MAC | 126 | −3.17 | +1.24 | — |
| BIRD 9B | C3 | 320 | +7.19 | +1.64 | 4.39× |
| BIRD 9B | DIN | 320 | +5.62 | +1.64 | 3.43× |
| 27B→9B | DIN | 633 | +21.48 | +8.95 | 2.40× |

**Solved queries are structurally trivial.** The 7–9 queries any method solves are all <400 characters with 0–1 JOINs, whereas the ≥5-JOIN, >3,500-character queries (≈70% of BEAVER) fail under every backbone and schema variant (predicted SQL ~30 chars against ~3,000-char gold—a code-generation, not schema-comprehension, failure). This caps ExAcc at 8–10%, driving refinement's contribution space toward zero; even MiniMax-M2.7's 4 committed refinements (§5.5) do not intersect the columns gold queries span. BEAVER thus marks the boundary between *schema-linking* (which EGRefine addresses) and *code-generation complexity* (which dominates here)—future work should check which regime its target benchmark falls in.

## 6.6 Query-Subset Decomposition: Concentration and Dilution

We partition each (benchmark, backbone, algorithm) cell into the *touching subset*—queries whose gold SQL references ≥1 refined column—and its complement, with the refined-column set fixed per (benchmark, backbone) so NoRef and EGRefine use identical partitions.

**Concentration.** On Dr.Spider-Abbr (27B), C3 and DIN-SQL show 4.5× concentration, peaking on the 27B→9B cross-model cell (Table 9). This is the query-level reading of the coverage–improvement linearity: modest aggregate Δ under limited coverage reflects the small touching fraction (≈22% on Dr.Spider-Abbr, ≈8% on BIRD), not weak per-query effect. Non-touching queries move little: zero by construction on Dr.Spider (reuse-mode) and ≤1.63 pp on BIRD (prompt-level noise).

**Algorithm-specific differential on BIRD.** On BIRD's touching subset (n=126), DIN-SQL gains while C3 and MAC-SQL do not benefit (Table 9); their positive aggregates come from non-touching queries. Per §6.4, column-name token shifts disrupt C3's schema-pruning and MAC-SQL's decomposition, while DIN-SQL's chain-of-thought prompting [50] is more robust.

## 7 CONCLUSION

EGRefine formalizes Text-to-SQL schema refinement as a constrained optimization solved through execution-grounded verification, yielding a durable, drop-in schema. Across Dr.Spider, BIRD, and BEAVER, execution feedback is load-bearing: it converts unreliable LLM preferences into dependable refinements, transfers across models, and degrades gracefully beyond current Text-to-SQL reach.

**Limitations.** The refined schema is specific to its (schema, $Q$, $\mathcal{M}$) triple: recovery varies by architecture (§6.4); it abstains on SQL-generation-bound schemas (BEAVER, §6.5); and its execution-grounded verification requires a representative labeled workload—natural-language questions paired with gold SQL—to score candidates, the standard assumption of workload-driven physical design [6, 45]. Relaxing this to weak supervision (executed query logs without gold annotations, or NL-only self-consistency) is future work.

## REFERENCES


[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[2] Sanjay Agrawal, Surajit Chaudhuri, and Vivek Narasayya. 2000. Automated Selection of Materialized Views and Indexes for SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases (VLDB)*.

[3] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. 2023. Benchmarking and Improving Text-to-SQL Generation under Ambiguity. In *EMNLP*. 7053–7074.

[4] Shuaichen Chang, Jun Wang, Mingwen Dong, et al. 2023. Dr.Spider: A Diagnostic Evaluation Benchmark Towards Text-to-SQL Robustness. *arXiv preprint arXiv:2301.08881* (2023).

[5] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. SQL-of-Thought: Multi-Agentic Text-to-SQL with Guided Error Correction. *arXiv preprint arXiv:2509.00581* (2025).

[6] Surajit Chaudhuri and Vivek Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*.

[7] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proceedings of the International Conference on Very Large Data Bases (VLDB)*. 3–14.

[8] Bei Chen, Fengji Zhang, Anh Nguyen, et al. 2023. CodeT: Code Generation with Generated Tests. *arXiv preprint arXiv:2207.10397* (2023).

[9] Kaiwen Chen, Yueting Chen, Xiaohui Yu, and Nick Koudas. 2025. Reliable Text-to-SQL with Adaptive Abstention. *arXiv preprint arXiv:2501.10858* (2025).

[10] Peter Baile Chen, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2024. BEAVER: An Enterprise Benchmark for Text-to-SQL. *arXiv preprint arXiv:2409.02038* (2024).

[11] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128* (2024).

[12] Ziru Chen, Shijie Chen, Michael White, Raymond Mooney, et al. 2023. Text-to-SQL Error Correction with Language Models of Code. In *ACL (Short Papers)*. 1359–1372.

[13] Xuemei Dong, Chao Zhang, Yuxiang Ge, Yun Mao, Yongkang Gao, Jian Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306* (2023).

[14] Ahmed Elgohary, Saghar Hosseini, and Ahmed Hassan Awadallah. 2020. Speak to Your Parser: Interactive Text-to-SQL with Natural Language Feedback. In *ACL*. 2065–2077.

[15] Jonathan Fürst, Catherine Kosten, Farhad Nooralahzadeh, et al. 2024. Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries. *arXiv preprint arXiv:2402.08349* (2024).

[16] Yujian Gan, Xinyun Chen, Qiuping Huang, Matthew Purver, John R. Woodward, Jinxia Xie, and Pengsheng Huang. 2021. Towards Robustness of Text-to-SQL Models against Synonym Substitution. In *ACL*. 2505–2515.

[17] Yujian Gan, Xinyun Chen, and Matthew Purver. 2021. Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization. In *EMNLP*. 8926–8931.

[18] Dawei Gao, Haibin Wang, Yaliang Li, et al. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint arXiv:2308.15363* (2023).

[19] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book* (2nd ed.). Pearson Prentice Hall.

[20] Zijin Hong, Zheng Yuan, Qinggang Zhang, et al. 2025. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2025).

[21] Klaus Jansen and Petra Scheffler. 1997. Generalized coloring for tree-like graphs. *Discrete Applied Mathematics* 75, 2 (1997), 135–155.

[22] George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A Survey on Deep Learning Approaches for Text-to-SQL. *The VLDB Journal* 32, 4 (2023), 905–936.

[23] Fangyu Lei et al. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *ICLR*.

[24] Haoyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, and Junliang Li. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proc. of AAAI*.

[25] Haoyang Li, Jing Zhang, Hanbing Liu, et al. 2024. CodeS: Towards Building Open-Source Language Models for Text-to-SQL. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[26] Jinyang Li, Binyuan Hui, Ge Qu, et al. 2023. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[27] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In *Findings of EMNLP*. 4870–4888.

[28] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural Language to SQL: State of the Art and Open Problems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5466–5471.

[29] Kyle Luoma and Arun Kumar. 2025. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–25.

[30] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. 2024. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. *arXiv preprint arXiv:2408.07702* (2024).

[31] Wuwei Mao et al. 2024. Enhancing Text-to-SQL Parsing through Question Rewriting and Execution-Guided Refinement. In *Findings of ACL*. 2009–2024.

[32] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *8th Biennial Conference on Innovative Data Systems Research (CIDR)*.

[33] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *NeurIPS*. 36339–36348.

[34] Luyu Qiu, Jianing Li, Chi Su, and Lei Chen. 2025. Interactive Text-to-SQL via Expected Information Gain for Disambiguation. *arXiv preprint arXiv:2507.06467* (2025).

[35] Sahil Qiu et al. 2025. PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries. In *NAACL*.

[36] Ge Qu, Jinyang Li, Bowen Qin, et al. 2025. SHARE: An SLM-Based Hierarchical Action Correction Assistant for Text-to-SQL. In *ACL*. 11268–11292.

[37] Cedric Renggli, Ihab F. Ilyas, and Theodoros Rekatsinas. 2025. Fundamental Challenges in Evaluating Text2SQL Solutions and Detecting Their Limitations. *arXiv preprint arXiv:2501.18197* (2025).

[38] Irina Saparina and Mirella Lapata. 2024. AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries. In *NeurIPS*. 90600–90628.

[39] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *EMNLP*. 9895–9901.

[40] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.

[41] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[42] Yuan Tian, Zheng Zhang, Zheng Ning, et al. 2023. Interactive Text-to-SQL Generation via Editable Step-by-Step Explanations. In *EMNLP*. 16149–16166.

[43] Kapil Vaidya, Abishek Sankararaman, Jialin Ding, Chuan Lei, Xiao Qin, Balakrishnan Narayanaswamy, and Tim Kraska. 2025. ODIN: A NL2SQL Recommender to Handle Schema Ambiguity. *arXiv preprint arXiv:2505.19302* (2025).

[44] Gary Valentin, Michael Zuliani, Daniel C. Zilio, Guy M. Lohman, and Alan Skelley. 2000. DB2 Advisor: An Optimizer Smart Enough to Recommend Its Own Indexes. In *Proceedings of the International Conference on Data Engineering (ICDE)*. 101–110.

[45] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-Scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 1009–1024.

[46] Bing Wang, Yan Gao, Zhoujun Li, and Jian-Guang Lou. 2023. Know What I Don't Know: Handling Ambiguous and Unknown Questions for Text-to-SQL. In *Findings of ACL*. 5701–5714.

[47] Bing Wang, Changyu Ren, Jian Yang, et al. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *COLING*. 540–557.

[48] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for

Text-to-SQL Parsers. In *ACL*. 7567–7578.

[49] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*.

[50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*. 24824–24837.

[51] Fabian Wenz, Omar Bouattour, Devin Yang, Justin Choi, Cecil Gregg, Nesime Tatbul, and Çağatay Demiralp. 2026. BenchPress: A Human-in-the-Loop Annotation System for Rapid Text-to-SQL Benchmark Curation. In *CIDR*.

[52] Liu Xinyu, Shen Shuyu, Li Boyan, et al. 2025. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? *arXiv preprint arXiv:2408.05109* (2025).

[53] Tao Yu, Rui Zhang, Heyang Er, et al. 2019. CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases. In *EMNLP-IJCNLP*. 1962–1979.

[54] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. 3911–3921.

[55] Meng Zhang, Kexin Ma, Liyang Xu, Kedi Zhang, Yuanxi Peng, and Ruochun Jin. 2025. CLEAR: A Parser-Independent Disambiguation Framework for NL2SQL. In *ICDE*. 1–14.

[56] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning. *arXiv preprint arXiv:1709.00103* (2017).